

-
- 

DEVPOST

-  Search hackathons... 
- - [Log in](#)
 - [Sign up](#)
- 

[Join a hackathon](#) ▾ ▴



[Devpost](#)

[Participate in our public hackathons](#)

[Hackathons](#) → [Projects](#) →

⟨D⟩

[Devpost for Teams](#)

[Access your company's private hackathons](#)

[Login](#) →

[Host a hackathon](#) ▾ ▴



[Devpost](#)

[Grow your developer ecosystem and promote your platform](#)

[Host a public hackathon](#) →

⟨D⟩

[Devpost for Teams](#)

[Drive innovation, collaboration, and retention within your organization](#)

[Host an internal hackathon](#) →

By use case

[AI hackathons](#) → [Customer hackathons](#) → [Employee hackathons](#) → [Public hackathons](#) →

[Resources](#) ▾ ▴

[Blog](#)

[Insights into hackathon planning and participation](#)

[Customer stories](#)

[Inspiration from peers and other industry leaders](#)

[Planning guides](#)

[Best practices for planning online and in-person hackathons](#)



[Webinars & events](#)[Upcoming events and on-demand recordings](#)[Help desk](#)[Common questions and support documentation](#)[D](#)[Join a hackathon](#) ▾ ▴[Back](#)[D](#)[Devpost](#)[Participate in our public hackathons](#)[Hackathons](#) → [Projects](#) →[D](#)[Devpost for Teams](#)[Access your company's private hackathons](#)[Login](#) →[Host a hackathon](#) ▾ ▴[Back](#)[D](#)[Devpost](#)[Grow your developer ecosystem and promote your platform](#)[Host a public hackathon](#) →[D](#)[Devpost for Teams](#)[Drive innovation, collaboration, and retention within your organization](#)[Host an internal hackathon](#) →

By use case

[AI hackathons](#) → [Customer hackathons](#) → [Employee hackathons](#) → [Public hackathons](#) →[Resources](#) ▾ ▴

- [Back](#)

- [Blog](#)

[Insights into hackathon planning and participation](#)

[Customer stories](#)

[Inspiration from peers and other industry leaders](#)

[Planning guides](#)

[Best practices for planning online and in-person hackathons](#)

[Webinars & events](#)

[Upcoming events and on-demand recordings](#)

[Help desk](#)

[Common questions and support documentation](#)

- 

-  

- [Log in](#)

- [Sign up](#)

Fieldscope

A local NLP platform that turns research papers into topic specific knowledge graphs and positions new papers within those scientific landscapes to create analysis on topical and study specific trends

[Like 1](#)

[Comment](#)

- [Story](#)

- [Updates](#)

03:01



Inspiration

When we started brainstorming ideas, we kept running into the same problem: most NLP hackathon projects felt like thin wrappers around existing language models. Paper summarizers, research chatbots, sentiment analyzers, and question answering tools are useful, but they are also things modern LLMs already do reasonably well with a couple prompts. We wanted to build something more analytical, a tool that uses computational linguistics to more quantitatively model the structure of scientific literature. Scientific research can be represented as claims, citations, methods, evidence, disagreements, and changes over time. We were interested in building a system that could represent those relationships directly. Our core motivation was that helping people understand scientific literature more clearly is a form of social good. Researchers, students, journalists, and policy analysts often need to understand not only what a paper says, but how it fits into a larger body of work. We wanted to build a tool that could help users reason about research areas at the level of claims, evidence, controversy, and methodological strength at both a qualitative and quantitative level that LLMs alone can't match.

What it does

Fieldscope is a local scientific literature analysis platform that turns collections of research papers into structured scientific landscapes. A user can create a landscape for a research area, upload individual papers or bulk upload multiple papers, and run analysis on that collection. The system parses each paper, extracts important claims, analyzes methodology and evidence, builds a graph of relationships between papers and claims, and generates landscape level insights.

The platform has two main workflows.

Scientific Landscape Building: Users create a topic-specific landscape, upload papers into it, and analyze the overall field. A landscape acts as a persistent collection of papers connected by claims, citations, methods, support relationships, contradictions, and semantic similarity.

Relational Paper Analysis: After a landscape has been built, users can upload a new candidate paper and analyze it in the context of that selected landscape. The candidate paper is compared against the existing research area rather than being analyzed in isolation.

The Relational Paper Analysis system calculates five main metrics:

- **Consensus Alignment Score (CAS):** Measures how strongly the candidate paper's claims align with existing literature.
- **Field Controversy Index (FCI):** Estimates how contested the surrounding research area is.
- **Methodological Standing Score (MSS):** Compares the candidate paper's methodology against related or contradicting papers.
- **Claim Novelty Score (CNS):** Measures how semantically different the candidate paper's claims are from existing claims.
- **Temporal Field Position (TFP):** Attempts to place the paper within the field's historical development using publication years and semantic drift.

The goal is to help users answer questions like: What does this paper claim? How strong is its evidence? Does it support or challenge existing work? Is the surrounding field settled or contested? Are its claims novel, or are they similar to previous work?

How we built it

The backend is built with Python and FastAPI. The frontend uses React, Vite, React Router, and Tailwind CSS. For document processing, the system supports PDF, XML, and HTML inputs. PDFs are parsed through GROBID when available, with a pdfminer fallback. HTML and XML files are parsed locally with BeautifulSoup and XML parsing tools. After parsing, papers are converted into structured sections such as title, abstract, introduction, methods, results, discussion, citations, and extracted claims. Claim extraction uses SciBERT embeddings along with rule-based scoring to identify claim-like sentences. Each claim receives text, polarity, confidence, source section, and an embedding. We also built a multiagent paper assessment layer. Different lightweight agents estimate stance, methodology, study design, sample size, quality, weaknesses, and uncertainty signals. These outputs are combined into broader paper-level assessments. The extracted paper information is stored in Neo4j as a knowledge graph. The graph contains nodes such as papers, claims, methods, citations,

and temporal snapshots. Relationships include connections such as "SUPPORTED_BY, USES_METHOD, CITES, SUPPORTS, and CONTRADICTS".

For graph-based analysis, we used PyTorch Geometric to define a heterogeneous graph neural network (GNN) that models relationships between papers, claims, and citations. The system also uses Sentence Transformers for semantic evolution analysis, estimating how concepts and claims shift over time. To make the system usable across multiple research areas, we built a local persistence system. Papers, processed outputs, and landscapes are stored locally under a data/ directory. Landscapes are stored as metadata collections that reference paper IDs, rather than as new Neo4j nodes. This lets the same graph architecture support multiple user-defined research areas without changing the underlying graph schema. The RPA page connects to this landscape system. A user selects an existing landscape, uploads a candidate paper, and receives relational metrics generated from backend analysis rather than placeholder data.

Challenges we ran into

One of the biggest challenges was defining the product clearly. We began by considering several simpler NLP ideas, but many of them felt too close to what an LLM could already do (ex. give a simple summary of a given field). We spent a lot of time narrowing the project into something more specific and quantitative, a platform for modeling scientific literature as a graph of claims, methods, evidence, and relationships.

Another major challenge was integration. The project combines document parsing, claim extraction, embeddings, graph databases, graph neural networks, local storage, frontend state management, and API routing. Each part had its own data format, and getting them to work together smoothly was difficult. Even using AI agents to patch connection issues within the backend was a tedious process.

Storage and landscape management also became more complex than expected. We needed users to be able to create multiple research landscapes, bulk-upload papers, persist those papers locally, and later select a landscape for RPA. We wanted this without changing the Neo4j schema or disrupting the existing graph pipeline, so we had to pivot to using json for landscape storage.

The RPA workflow was especially challenging because it depends on a correctly populated landscape. The candidate paper has to be processed, inserted into the graph, and compared only against papers in the selected landscape without contaminating its own comparison "corpus".

Accomplishments that we're proud of

We're proud that we've built a system that goes beyond summarization (and hopefully is as accurate as we say it is 🤖). Instead of only producing natural language outputs, Fieldscope creates structured representations of research areas. It extracts claims, stores relationships, builds graphs, and generates quantitative metrics about consensus, controversy, methodology, novelty, and temporal position. We are also proud of the two step product model we developed: build a landscape, then position a new paper within it. That framework made the project much clearer and gave the RPA system a real reference corpus instead of analyzing papers in isolation. Another accomplishment was getting multiple complex pieces to work together in one local application. The final system connects a React frontend, FastAPI backend, local file persistence, paper registry, landscape storage, Neo4j graph construction, NLP models, graph analysis, and RPA metrics. Most importantly, we are proud that we were able to think of and build a product that stays rooted in a real problem and solves it in a way that, with more development, could outperform modern tools like LLMs. Scientific literature is difficult to navigate, especially when fields are controversial or rapidly evolving. Our project tries to make that structure easier to inspect.

What we learned

We learned that the hardest part of building NLP tools is designing the right workflow for the model that combines multiple analytical strategies. We also learned how difficult scientific document processing can be. Papers vary widely in formatting, metadata quality, section structure, and citation structure. Even before analysis begins, turning a paper into clean structured data is a major challenge (one that we haven't entirely mastered yet 🤖). On the technical side, we gained more experience with FastAPI, React, Neo4j, PyTorch Geometric, SciBERT, Sentence Transformers, local persistence, API design, and graph-based/GNN analysis. A major lesson was the importance of scoping. We had to repeatedly cut or simplify ideas so the final product would be coherent, and the strongest version of the project came from focusing on one clear product loop: create a landscape, then analyze a paper within that landscape.

What's next for Fieldscope

The next step is to make the analysis more scientifically robust, as right now, some parts of the system are still experimental. The GNN is defined and integrated, but it does not yet use a trained project-specific checkpoint. Some stance and semantic

evolution logic is still specialized to early test domains, and publication year extraction needs to be implemented.

Future work would include training and validating the graph models on larger scientific corpora, improving claim-to-claim support and contradiction detection, expanding beyond the initial domains, and making temporal analysis more reliable. Most of this comes with creating and implementing public landscapes that are built upon by all users in order to improve model complexity. We also want to create the user experience for exploring landscapes visually, as a stronger graph viewer could help users inspect clusters of papers, controversial claims, highly connected studies, and areas where the literature is still developing.

Ultimately, we see Fieldscope as a step toward tools that help researchers, students, journalists, and analysts understand scientific fields as connected landscapes, all through the power of computational linguistics.

Built With

- [beautiful-soup](#)
- docker-compose
- fastapi
- grobit
- [neo4j](#)
- pdfminer.six
- pydantic
- [python](#)
- [pytorch](#)
- [react](#)
- scibert
- sentence-transformers
- tailwind
- vite

Try it out

- [GitHub Repo](#)

Submitted to



[LingHacks VII](#)

- Winner Top 10 Teams
- Winner All Winners

Created by

- I worked on both the front and backend of the site. I also created the video.



[Siva Ram](#)



- I worked on frontend and backend for the Relational Paper Analysis and made the writeups and descriptions



[Dhruv Avirneni](#)



[Like 1](#)

1 person likes this:



•

Share this project:

-
-
-

Updates



[Dhruv Avirneni](#) started this project — [3 days ago](#)

Leave feedback in the comments!

[Log in](#) or [sign up for Devpost](#) to join the conversation.

Devpost

- [About](#)
- [Careers](#)
- [Contact](#)
- [Help](#)

Hackathons

- [Browse hackathons](#)
- [Explore projects](#)
- [Host a hackathon](#)
- [Hackathon guides](#)

Portfolio

- [Your projects](#)
- [Your hackathons](#)
- [Settings](#)

Connect

-  [Twitter](#)
-  [Discord](#)
-  [Facebook](#)
-  [LinkedIn](#)

© 2026 Devpost, Inc. All rights reserved.

- [Community guidelines](#)
- [Security](#)
- [CA notice](#)
- [Privacy policy](#)
- [Terms of service](#)